

Machine Learning

Lecture 4: Discriminative and Generative Classification

A textbook treatment

Based on the lecture notes and slides of Mounir Ghogho

September 25, 2026

Contents

How to read this book	2
4 Discriminative and Generative Classification	3
How to read the formulas in this chapter	3
4.1 Two Routes to the Posterior	3
4.2 The Discriminative Route	5
4.3 The Generative Route	9
4.4 Gaussian Class-Conditional Models	12
4.5 Naive Bayes	16
4.6 Comparing the Two Routes	20
4.7 Class Imbalance and Prior Shift	21
4.8 Unfamiliar Inputs and Combined Systems	23
4.9 Model Choice and Lab Preparation	24
Notes on the original slides and notes	25
Chapter takeaways	25

How to read this book

Each concept in this book is presented in the same rhythm.

- 1. Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
- 2. Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
- 3. Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
- 4. Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
- 5. Examples.** From “trivially small” to “genuinely useful.”
- 6. Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, “say it out loud” version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Basic probability (random variables, conditional distributions, expectations), vectors in $\mathbb{R}^d$, and a first course in calculus. The information-theoretic quantities used (entropy, cross-entropy, Kullback–Leibler divergence) are defined and proved from scratch where they appear.

Chapter 4

Discriminative and Generative Classification

How to read the formulas in this chapter

Chapter 2 taught you to read $\mathbb{P}(Y \mid X = x)$, losses and risks. This chapter adds a few symbols, all listed here; as before, every formula is followed by a *Reading the formula* box.

Symbol	Meaning in words
$p(y = c \mid x)$	the <i>posterior</i> : probability of class c once x is seen (the true one; p_θ or $\hat{p}$ for a model's estimate)
$s_{\theta,c}(x)$	a real-valued <i>score</i> (logit) that a model gives to class c
$\sigma(s)$	the <i>sigmoid</i> , which turns one score into a probability in $(0, 1)$
$\text{softmax}(s)_c$	turns C scores into C probabilities that sum to 1
$\pi_c = p(y = c)$	the <i>prior</i> : how frequent class c is before seeing any input
$p_\theta(x \mid y = c)$	the <i>class-conditional</i> distribution: which inputs class c produces (a density for continuous x , a mass function for discrete x)
$\phi = (\pi, \theta)$	all parameters of a generative model: priors plus class-conditional parameters
$\mathcal{N}(x; \mu, \Sigma)$	the Gaussian density with mean μ and covariance Σ
N_c	number of training examples of class c ; $N = \sum_c N_c$
n_{cj}	count of token (word) j in the training documents of class c
α	a smoothing constant (pseudo-count) added to every count
π_t, π_d	class priors at training time and at deployment time
$\mathbf{1}(\cdot), \log$	indicator; natural logarithm unless stated

4.1 Two Routes to the Posterior

What are we trying to get?

Chapter 2 showed that once the posterior $p(y = c \mid x)$ is known, the best decision is a one-line rule. So the real work of classification is *estimating the posterior from labelled examples*. We want to understand that there are two fundamentally different ways to do that, what each one commits us to, and why neither wins in general.

The decision is known; the posterior is not

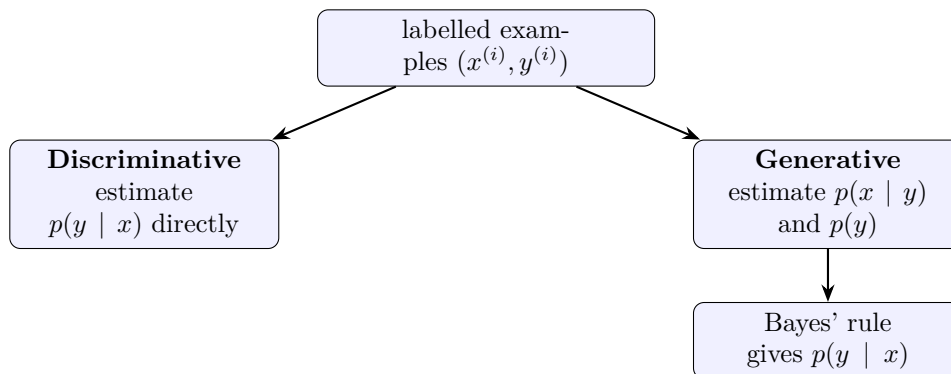
Let $Y \in \{1, \dots, C\}$ be a class label and $X \in \mathbb{R}^d$ a feature vector. If the posterior were known, Theorem 2.2.3 says to choose the action with the smallest conditional expected loss,

$$f^*(x) \in \arg \min_{a \in \mathcal{A}} \mathbb{E}[\ell_{\text{dec}}(Y, a) \mid X = x],$$

and under zero-one loss this is $f^*(x) = \arg \max_c p(y = c \mid x)$ (Theorem 2.2.8). The learning problem arises because $p(y = c \mid x)$ is unknown and must be estimated from labelled examples $\mathcal{D}_N = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$: for instance, is this network flow benign or malicious?

The Bayes rule is easy. The hard part is getting the probabilities it needs.

- Definition 4.1.1** (Discriminative and generative classifiers). • A *discriminative* model directly approximates the posterior with a parameterized family $p_\theta(y = c \mid x)$; training produces the fitted posterior $p_{\hat{\theta}}(y = c \mid x)$. It learns *where the label changes* as a function of the features.
- A *generative* classifier models the class prior $p(y = c)$ and the class-conditional distribution $p(x \mid y = c)$, and obtains the posterior through Bayes' rule. It learns *what each class looks like*, then compares how well each class explains the observed input.



Reading the formula

Both routes end with the same object, a posterior over classes. The difference is the *direction of modelling*. Discriminative: “given these features, what is the probability of attack?” Generative: “what do attack flows typically look like, what do benign flows look like, and how common is each? Now, which story fits this flow better?” The second route builds two “worlds” and asks which one this input belongs to.

- Example 4.1.2** (Where each route shows up). • *Spam filtering.* Naive Bayes (generative) models the token frequencies of spam and of legitimate mail. It is cheap to estimate and still serves as a baseline or a first triage layer.
- *Fraud and intrusion screening.* Class-conditional models score how *compatible* an event is with normal behaviour and with known malicious behaviour, supporting both classification and anomaly triage. Modern systems combine this with richer discriminative and behavioural signals.

- *Image classification.* Deep discriminative networks dominate: modelling the full density of images per class is far harder than learning where the label changes.

Remark 4.1.3 (“Generative” is about direction, not task). A class-conditional model $p(x \mid y = c)$ can be used to *infer* the class of an observed x (classification), or, in principle, to *draw* a new x for a chosen class (generation). Classical generative classifiers and modern generative AI share this modelling direction; their model families and scales differ enormously.

Neither route is universally better. Their relative value depends on the amount of data, the credibility of the modelling assumptions, the representation, and the deployment objective. The rest of the chapter makes each route concrete and then compares them.

Summary: Two routes

- Decision rule known (Bayes); posterior unknown; must be learned from $\mathcal{D}_N$.
- Discriminative: model $p(y \mid x)$. Generative: model $p(x \mid y)$ and $p(y)$, then apply Bayes’ rule.
- Same target, different commitments; which is better depends on data, assumptions and objective.

4.2 The Discriminative Route

What are we trying to get?

We want to see how a model that outputs *numbers* (scores) is turned into *probabilities* and then into *actions*, why the cross-entropy loss of Chapter 2 is exactly maximum likelihood here, and what a discriminative model cannot tell us.

Scores, probabilities, actions

A discriminative model often first produces a real-valued *score* (or *logit*) for each class. Scores are converted into posterior estimates and then into an action:

$$\{s_{\hat{\theta},c}(x)\} \longrightarrow \{p_{\hat{\theta}}(y = c \mid x)\} \longrightarrow \hat{y}(x).$$

The score model determines how inputs are *ranked*; the probability transformation quantifies *relative uncertainty* among classes; the decision loss determines the *action*.

Definition 4.2.1 (Sigmoid; binary logistic model). The *sigmoid* (logistic) function is $\sigma(s) = \frac{1}{1 + e^{-s}}$. For binary classification one score suffices, and the posterior estimate is

$$p_{\hat{\theta}}(y = 1 \mid x) = \sigma(s_{\hat{\theta}}(x)) = \frac{1}{1 + \exp[-s_{\hat{\theta}}(x)]}.$$

Logistic regression is the special case of a linear score, $s_{\theta}(x) = w^{\top}x + b$, $\theta = (w, b)$.

Reading the formula

σ squashes any real number into $(0, 1)$: $\sigma(0) = 0.5$, $\sigma(2) \approx 0.88$, $\sigma(-2) \approx 0.12$, $\sigma(5) \approx 0.993$. A score of 0 means “no idea”; large positive means “confidently class 1”; large negative means “confidently class 0.” With $w = (1, -2)$, $b = 0.5$ and $x = (2, 1)$: $s = 2 - 2 + 0.5 = 0.5$ and $p(y = 1 | x) = \sigma(0.5) \approx 0.62$.

Proposition 4.2.2 (Properties of the sigmoid). *For all $s \in \mathbb{R}$: (i) $0 < \sigma(s) < 1$; (ii) $\sigma(-s) = 1 - \sigma(s)$; (iii) σ is strictly increasing; (iv) $\sigma(s) > \frac{1}{2} \iff s > 0$; (v) more generally, for $t \in (0, 1)$, $\sigma(s) > t \iff s > \log \frac{t}{1-t}$.*

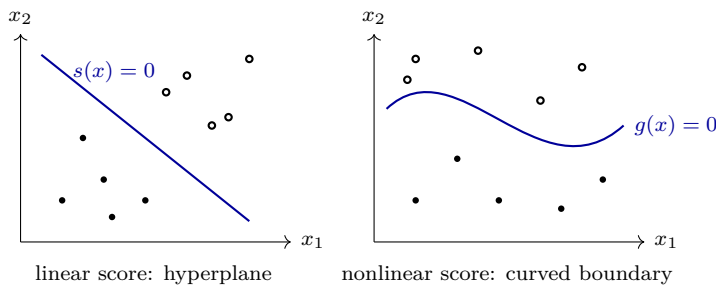
Proof. (i) $e^{-s} > 0$ so the denominator exceeds 1. (ii) $1 - \sigma(s) = \frac{e^{-s}}{1 + e^{-s}} = \frac{1}{e^s + 1} = \sigma(-s)$ after multiplying numerator and denominator by e^s . (iii) e^{-s} is strictly decreasing, so $1 + e^{-s}$ is, so its reciprocal is strictly increasing. (v) $\sigma(s) > t \iff 1 + e^{-s} < 1/t \iff e^{-s} < (1-t)/t \iff -s < \log \frac{1-t}{t} \iff s > \log \frac{t}{1-t}$. (iv) is (v) with $t = \frac{1}{2}$, since $\log 1 = 0$. $\square$

Theorem 4.2.3 (Decision boundaries of score models). *Under zero-one loss, the binary logistic model decides $\hat{y}(x) = 1 \iff \sigma(s_{\hat{\theta}}(x)) > \frac{1}{2} \iff s_{\hat{\theta}}(x) > 0$. For logistic regression the 0.5-posterior contour is the hyperplane $\hat{w}^\top x + \hat{b} = 0$. Under costs C_{FP}, C_{FN} (Theorem 2.2.9), the decision is $\hat{y}(x) = 1 \iff s_{\hat{\theta}}(x) > \log \frac{C_{FP}}{C_{FN}}$. For a nonlinear score $g_{\hat{\theta}}$, the boundary is the surface $g_{\hat{\theta}}(x) = 0$ (shifted by the same constant under unequal costs).*

Proof. The zero-one Bayes decision compares $p(y = 1 | x)$ with $\frac{1}{2}$ (Theorem 2.2.8), and Proposition 4.2.2(iv) converts that into $s > 0$; for a linear score, $s = 0$ is a hyperplane. Under costs, Theorem 2.2.9 gives threshold $t = C_{FP}/(C_{FP} + C_{FN})$, and Proposition 4.2.2(v) with $\frac{t}{1-t} = \frac{C_{FP}}{C_{FN}}$ gives $s > \log(C_{FP}/C_{FN})$. $\square$

Reading the formula

The probability threshold does *not* make the geometry linear; the score family does. A linear score gives a straight boundary; a curved score gives a curved boundary; changing the costs only *slides* the boundary along the score axis. Cyberattack example with $C_{FN} = 10$, $C_{FP} = 1$: the boundary moves from $s = 0$ to $s = \log(1/10) \approx -2.3$, i.e. raise the alarm already when the score is mildly negative. Expensive missed attacks lower the threshold; expensive false alarms raise it. One probability model supports several policies.



Multiclass scores and softmax

Definition 4.2.4 (Softmax). For scores $s = (s_1, \dots, s_C)$, the *softmax* map produces

$$p_\theta(y = c \mid x) = \text{softmax}(s_\theta(x))_c = \frac{\exp[s_{\theta,c}(x)]}{\sum_{k=1}^C \exp[s_{\theta,k}(x)]}, \quad c = 1, \dots, C.$$

Reading the formula

Exponentiate to make every score positive, then divide by the total so that the classes compete for probability one. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$. The largest score always gets the largest probability, and differences of scores are what matter: adding 100 to every score multiplies numerator and denominator by e^{100} and changes nothing.

Proposition 4.2.5 (Softmax facts). (i) The output lies in the simplex Δ^{C-1} and $\arg \max_c p_\theta(y = c \mid x) = \arg \max_c s_{\theta,c}(x)$. (ii) Adding the same constant to every score leaves all probabilities unchanged, so only $C - 1$ score contrasts are identifiable. (iii) With $C = 2$, $\text{softmax}(s)_1 = \sigma(s_1 - s_2)$: softmax reduces to the sigmoid of the score difference.

Proof. (i) Exponentials are positive and the entries are divided by their sum, so they are nonnegative and sum to 1; exp is increasing, so the ordering of scores is the ordering of probabilities. (ii) Replacing s_k by $s_k + a$ multiplies every exponential by e^a , which cancels between numerator and denominator. (iii)
$$\frac{e^{s_1}}{e^{s_1} + e^{s_2}} = \frac{1}{1 + e^{s_2 - s_1}} = \sigma(s_1 - s_2). \quad \square$$

Interpretation & discussion

Implementations still produce C scores rather than $C - 1$: every class then plays the same role, relabelling merely permutes outputs, and one vectorised output layer and loss serve any C . A reference class can remove the redundancy when identifiability matters (e.g. for interpreting coefficients).

Cross-entropy is conditional maximum likelihood

For example i , cross-entropy penalises the probability assigned to the true class:

$$\ell_{\text{CE}}^{(i)}(\theta) = - \sum_{c=1}^C \mathbf{1}(y^{(i)} = c) \log p_\theta(y = c \mid x^{(i)}) = - \log p_\theta(y^{(i)} \mid x^{(i)}),$$

so probabilities 0.9, 0.5, 0.1 on the true class cost about 0.11, 0.69, 2.30: confident mistakes are punished hardest.

Proposition 4.2.6 (Cross-entropy equals negative conditional log-likelihood). Assuming the labels are conditionally independent given their inputs, the conditional likelihood is $L_{\text{cond}}(\theta) = \prod_{i=1}^N p_\theta(y^{(i)} \mid x^{(i)})$, and

$$-\log L_{\text{cond}}(\theta) = - \sum_{i=1}^N \log p_\theta(y^{(i)} \mid x^{(i)}) = N \hat{R}_{\text{CE}}(\theta).$$

Minimising empirical cross-entropy is therefore the same as maximising the conditional likelihood.

Proof. Independence turns the joint probability of the observed labels into a product; the logarithm turns the product into a sum; the sum divided by N is the empirical cross-entropy of Definition 2.5.8. Since $-\log$ is decreasing, maximising L_{cond} and minimising $\hat{R}_{\text{CE}}$ have the same solutions. $\square$

Reading the formula

“Conditional” likelihood means we only ask the model to explain the *labels given the inputs*, never the inputs themselves. Three examples with true-class probabilities 0.9, 0.5, 0.1: likelihood $0.9 \times 0.5 \times 0.1 = 0.045$; negative log $= 0.105 + 0.693 + 2.303 = 3.10$; empirical cross-entropy $3.10/3 = 1.03$. Making the likelihood as large as possible and the average loss as small as possible are the same instruction.

Deep discriminative classifiers

Definition 4.2.7 (Feed-forward classifier). A neural network computes a sequence of representations $h^{(0)} = x$, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$ for $\ell = 1, \dots, L-1$, with φ an elementwise nonlinearity; the final layer produces scores $s_\theta(x) = W_L h^{(L-1)} + b_L$, and softmax produces $p_\theta(y = c \mid x) = \text{softmax}(s_\theta(x))_c$. Cross-entropy can train all layers end to end.

Reading the formula

Each layer is “multiply by a matrix, add a vector, bend with φ .” Logistic regression is the case $L = 1$: no hidden layers, scores linear in x . Adding layers lets the score be a curved function of x , so the boundary of Theorem 4.2.3 can curve. The representation the network learns is optimised for predicting the label, not for reconstructing or explaining the input distribution, which is exactly the point of the next subsection.

What the discriminative objective does not provide

A discriminative classifier focuses on predicting Y from X . This focus is usually an advantage, but it has consequences:

- the model need not learn whether an input x is *plausible*;
- a large $\max_c p_\theta(y = c \mid x)$ does not imply that x resembles the training data;
- the model provides no direct mechanism for sampling inputs or for marginalising missing features.

Softmax probabilities are *relative* comparisons among the modelled classes. If all classes explain an unfamiliar input poorly, one class can still receive a posterior near one.

A discriminative model ranks the classes it knows. It never asks whether the input makes sense.

Example 4.2.8 (Confident about nonsense). A three-class network trained on cat, dog and horse photographs receives a photograph of a toaster. Its scores come out (4.0, 0.5, −1.0), so softmax gives (0.96, 0.03, 0.01): “cat, 96%.” Nothing in the training objective ever asked the network “is this even an animal?” The number 0.96 measures how much better “cat” explains the toaster

than “dog” and “horse” do, which is all softmax can measure. Section 4.8 adds the missing check.

Remark 4.2.9 (Class imbalance is a different issue). If training and deployment share the same distribution, an imbalanced training prior is not an error: the fitted posterior *should* reflect that prevalence, and the boundary shifts accordingly (with a 5% positive class the 0.5 contour moves toward the positives). The practical difficulty is that severe imbalance may leave too few minority examples to estimate a reliable boundary. Weighting or resampling can help in that small-sample regime; with enough minority examples their benefit is limited and they may distort probability calibration. Section 4.7 separates this from a genuine *change* of prevalence between training and deployment.

Technique: Working with scores and probabilities

- To decide, compare the *score* with $\log(C_{\text{FP}}/C_{\text{FN}})$; you never need to compute σ .
- To convert a probability back to a score: $s = \log \frac{p}{1-p}$ (the *log-odds*); $p = 0.9$ gives $s \approx 2.2$.
- Softmax by hand: subtract the largest score from all scores first (Proposition 4.2.5(ii)); it avoids huge exponentials and changes nothing.
- Never read $\max_c p(y = c \mid x)$ as “probability that the input is normal.”

Summary: The discriminative route

- Scores $\rightarrow$ sigmoid/softmax $\rightarrow$ probabilities $\rightarrow$ decision; the score family fixes the boundary geometry, the costs only shift it (Theorem 4.2.3).
- Softmax is shift-invariant: $C - 1$ contrasts matter; with $C = 2$ it is the sigmoid of a difference.
- Cross-entropy = negative conditional log-likelihood (Proposition 4.2.6).
- Discriminative models do not model input plausibility; softmax confidence is relative.

4.3 The Generative Route

What are we trying to get?

We want a model that tells a *story* of how a labelled example came to be, to see how Bayes’ rule reverses that story into a classifier, and to understand why fitting such a model asks more of the data than the discriminative route does.

A probabilistic story for labelled data

Definition 4.3.1 (Generative classifier). A generative classifier imagines that an observation is produced in two steps: first choose a class, then generate features from that class,

$$Y \sim \text{Categorical}(\pi), \quad X \mid (Y = c) \sim p_{\theta}(\cdot \mid y = c),$$

with parameters $\phi = (\pi, \theta)$, $\pi_c = p(y = c)$. The joint model factorises as

$$p_\phi(x, y = c) = \pi_c p_\theta(x | y = c).$$

The *prior* π_c is the class frequency before the input is observed; the *class-conditional* density (or mass function) describes which inputs are plausible within class c .

Theorem 4.3.2 (Bayes' rule for classification). *Under the model of Definition 4.3.1, the posterior is*

$$p_\phi(y = c | x) = \frac{\pi_c p_\theta(x | y = c)}{\sum_{k=1}^C \pi_k p_\theta(x | y = k)}.$$

Classification under zero-one loss therefore chooses the class with the largest prior-weighted likelihood $\pi_c p_\theta(x | y = c)$.

Proof. By definition of conditional probability, $p(y = c | x) = p(x, y = c)/p(x)$. The numerator is $\pi_c p_\theta(x | y = c)$ by the factorisation. The denominator is the marginal $p(x) = \sum_k p(x, y = k) = \sum_k \pi_k p_\theta(x | y = k)$ (sum over the mutually exclusive, exhaustive classes). The denominator does not depend on c , so the $\arg \max$ over c is decided by the numerator. $\square$

Reading the formula

Numerator: “how common is class c ” times “how well does class c explain this x .” Denominator: the same quantity summed over all classes, which just rescales so that the posteriors add to 1. A rare class can still win if it explains the input much better. Network security: benign flows have prior 0.99 but give this flow likelihood 0.001; attacks have prior 0.01 but likelihood 0.20.

class	π_c	$p(x y = c)$	$\pi_c p(x y = c)$
benign	0.99	0.001	0.00099
attack	0.01	0.20	0.00200

$p(y = \text{attack} | x) = 0.002/(0.002 + 0.00099) \approx 0.67$. Attacks remain rare in the population, yet this particular flow is more compatible with the attack model, and the posterior says so.

Generative maximum likelihood

Proposition 4.3.3 (The generative likelihood splits). *For independent labelled observations, the joint likelihood is $L(\phi) = \prod_{i=1}^N p_\phi(x^{(i)}, y^{(i)})$ and its negative logarithm is*

$$\begin{aligned} -\log L(\phi) &= -\sum_{i=1}^N \left[\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} | y^{(i)}) \right] \\ &= \underbrace{-\sum_{c=1}^C N_c \log \pi_c}_{\text{prior part}} - \underbrace{\sum_{c=1}^C \sum_{i: y^{(i)}=c} \log p_\theta(x^{(i)} | y = c)}_{\text{one term per class}}. \end{aligned}$$

Hence the prior and each class-conditional model can be fitted separately: the

prior from the class counts alone, and the model of class c from the examples of class c alone.

Proof. Take logs of the product and substitute the factorisation $p_\phi(x, y) = \pi_y p_\theta(x | y)$. Group the prior terms by class: the label c occurs N_c times, giving $N_c \log \pi_c$. Group the class-conditional terms by class likewise. The first group depends only on π , the second only on θ , and within the second each class's parameters appear only in that class's sum. $\square$

Proposition 4.3.4 (Maximum-likelihood prior). *The prior part is maximised, over the simplex, by the empirical class frequencies $\hat{\pi}_c = N_c/N$.*

Proof. Write $q_c = N_c/N$, a probability vector. Then $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N H(q, \pi)$, the negative cross-entropy of Definition 1.8.1. By Gibbs' inequality (Theorem 1.8.2), $H(q, \pi) \geq H(q)$ with equality iff $\pi = q$. So the prior part is largest exactly at $\pi = q$. $\square$

Key insight

Fitting the prior is Gibbs' inequality again: the cross-entropy between the empirical class distribution and the model prior is minimised when they coincide. Generative training asks each example to be explained *twice*: its class (through π) and its features (through $p_\theta(x | y)$). That is a stronger requirement than the discriminative route's "explain the class given the features," and it is both the strength (more structure, less data needed) and the weakness (more assumptions that can be wrong) of the generative route.

Interpretation & discussion

Two approximations should be kept separate. *Shape approximation*: the family $p_\theta(x | y)$ restricts the class-conditional distributions and may not contain the truth; this error does not vanish with more data. *Finite-sample estimation*: even within the family, $\hat{\phi}$ differs from the best population parameter because it is estimated from finite data; this error shrinks with more data. Both make the fitted posterior $p_{\hat{\phi}}(y = c | x)$ differ from the true $p(y = c | x)$.

Summary: The generative route

- Story: choose a class from π , generate features from $p_\theta(x | y)$; joint $= \pi_c p_\theta(x | y = c)$.
- Bayes' rule turns the story into a posterior; classification compares prior-weighted likelihoods (Theorem 4.3.2).
- The likelihood splits by class (Proposition 4.3.3); the MLE prior is the class frequency (Proposition 4.3.4).
- Errors come from a wrong family (persistent) and from finite data (shrinking).

4.4 Gaussian Class-Conditional Models

What are we trying to get?

For continuous features, the simplest class “world” is a bell-shaped cloud. We want the fitted parameters in closed form, one worked posterior computed by hand, and, most importantly, to see how an *assumption about covariance* becomes a *shape of the decision boundary*: shared covariance gives straight boundaries (LDA), separate covariances give curved ones (QDA).

Definition 4.4.1 (Multivariate Gaussian class model). For $x \in \mathbb{R}^d$, the Gaussian class-conditional model is

$$p_\theta(x | y = c) = \mathcal{N}(x; \mu_c, \Sigma_c) = \frac{1}{(2\pi)^{d/2} |\Sigma_c|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c) \right],$$

with mean $\mu_c \in \mathbb{R}^d$ and symmetric positive-definite covariance Σ_c . In one dimension this is $\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp[-(x - \mu)^2/(2\sigma^2)]$.

Reading the formula

The mean μ_c is the centre of the cloud of class c . The covariance Σ_c is its shape: a multiple of the identity gives circular contours; unequal diagonal entries stretch the circle into an axis-aligned ellipse; off-diagonal entries rotate it (correlated features). The quadratic form $(x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c)$ is the *squared Mahalanobis distance*: “how many standard deviations away from the centre, in the direction of x .” The prefactor with $|\Sigma_c|$ makes the density integrate to one; a diffuse class (large determinant) spreads its probability thin.

Closed-form maximum likelihood

Theorem 4.4.2 (Gaussian MLE). Let N_c be the number of training examples of class c . The maximum-likelihood estimates are

$$\hat{\pi}_c = \frac{N_c}{N}, \quad \hat{\mu}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} x^{(i)}, \quad \hat{\Sigma}_c = \frac{1}{N_c} \sum_{i: y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top.$$

No iterative optimisation is needed.

Proof. The prior is Proposition 4.3.4. By Proposition 4.3.3 each class is fitted from its own examples, so fix a class and drop the index c ; write its $n = N_c$ examples as $x^{(1)}, \dots, x^{(n)}$. We give the full proof in one dimension; the multivariate case follows the same two steps with vector calculus. The negative log-likelihood is

$$J(\mu, \sigma^2) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^n (x^{(i)} - \mu)^2.$$

Step 1 (mean). For fixed σ^2 , $\partial J / \partial \mu = -\frac{1}{\sigma^2} \sum_i (x^{(i)} - \mu) = 0$ gives $\mu = \frac{1}{n} \sum_i x^{(i)}$, and J is a convex quadratic in μ , so this is the minimum. **Step 2 (variance).** With $\mu = \hat{\mu}$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, put $v = \sigma^2$: $\partial J / \partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $v = S/n$; the derivative is negative for $v < S/n$ and positive beyond, so it is the minimum. In d dimensions the same computation with Σ in place of v gives $\hat{\mu}$ as the class mean and $\hat{\Sigma}$ as the average outer product of the centred examples. $\square$

Remark 4.4.3 (N_c versus $N_c - 1$). The denominator N_c is the maximum-likelihood denominator. The familiar *unbiased* sample covariance uses $N_c - 1$. For classification the difference is immaterial when N_c is large, but the two are different estimators and you should say which one you use.

Example 4.4.4 (One dimension, by hand). Class 0: $\{0, 1, 2, 0, 1, 2\}$; class 1: $\{3, 4, 5\}$. Then $\hat{\pi}_0 = 6/9 = 2/3$, $\hat{\pi}_1 = 1/3$; $\hat{\mu}_0 = 6/6 = 1$, $\hat{\mu}_1 = 12/3 = 4$; $\hat{\sigma}_0^2 = (1 + 0 + 1 + 1 + 0 + 1)/6 = 2/3$, $\hat{\sigma}_1^2 = (1 + 0 + 1)/3 = 2/3$.

At $x = 2$: the two densities share the prefactor $1/\sqrt{2\pi \cdot 2/3}$, so compare exponentials. Class 0: $\exp[-(2 - 1)^2/(2 \cdot \frac{2}{3})] = \exp(-0.75) \approx 0.472$; class 1: $\exp[-(2 - 4)^2/(2 \cdot \frac{2}{3})] = \exp(-3) \approx 0.050$. Prior-weighted: $\frac{2}{3}(0.472) = 0.315$ and $\frac{1}{3}(0.050) = 0.017$. Normalising,

$$p_{\hat{\phi}}(y = 0 \mid x = 2) = \frac{0.315}{0.315 + 0.017} \approx 0.950, \quad p_{\hat{\phi}}(y = 1 \mid x = 2) \approx 0.050.$$

The point $x = 2$ is one unit from the centre of class 0 and two units from the centre of class 1; both the distance and the larger prior of class 0 push the posterior toward class 0.

Example 4.4.5 (Where is the boundary in Example 4.4.4?). With equal variances the log prior-weighted likelihoods are $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2 \cdot \frac{2}{3})$. Setting the two equal: $\log \frac{2}{3} - \frac{3}{4}(x - 1)^2 = \log \frac{1}{3} - \frac{3}{4}(x - 4)^2$, i.e. $\frac{3}{4}[(x - 4)^2 - (x - 1)^2] = \log \frac{1}{3} - \log \frac{2}{3} = -\log 2$, i.e. $\frac{3}{4}(15 - 6x) = -0.693$, giving $x \approx 2.65$. With equal priors the boundary would sit at the midpoint 2.5; the larger prior of class 0 pushes it toward class 1 by 0.15. This is LDA in one dimension: a single threshold.

Scores, LDA and QDA

Definition 4.4.6 (Generative class score). Define $s_{\hat{\phi},c}(x) := \log \hat{\pi}_c + \log p_{\hat{\theta}}(x \mid y = c)$. Then, by Theorem 4.3.2,

$$\log p_{\hat{\phi}}(y = c \mid x) = s_{\hat{\phi},c}(x) - \log \sum_{k=1}^C \exp[s_{\hat{\phi},k}(x)],$$

so the fitted posterior is the softmax of the generative scores, and class prediction can be based on the scores alone (the log-normaliser is common to all classes).

Theorem 4.4.7 (LDA: shared covariance gives linear scores). Assume every class has the same covariance, $\hat{\Sigma}_c = \hat{\Sigma}$. Then, up to a term common to all classes,

$$s_{\hat{\phi},c}(x) = x^\top \hat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c,$$

which is affine in x . Consequently the boundary between any two classes, $\{x : s_{\hat{\phi},c}(x) = s_{\hat{\phi},k}(x)\}$, is a hyperplane. This is Linear Discriminant Analysis.

Proof. Take the log of the Gaussian density:

$$\log p(x \mid y = c) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\hat{\Sigma}| - \frac{1}{2} (x - \hat{\mu}_c)^\top \hat{\Sigma}^{-1} (x - \hat{\mu}_c).$$

The first two terms do not depend on c (shared covariance). Expand the quadratic form:

$$(x - \hat{\mu}_c)^\top \hat{\Sigma}^{-1} (x - \hat{\mu}_c) = x^\top \hat{\Sigma}^{-1} x - 2x^\top \hat{\Sigma}^{-1} \hat{\mu}_c + \hat{\mu}_c^\top \hat{\Sigma}^{-1} \hat{\mu}_c,$$

using symmetry of $\hat{\Sigma}^{-1}$ to combine the two cross terms. The term $x^\top \hat{\Sigma}^{-1} x$ is common to all classes and drops out of any comparison; what remains, times $-\frac{1}{2}$, plus $\log \hat{\pi}_c$, is the displayed score. Two affine functions of x are equal on a hyperplane. $\square$

Key insight

The only x -quadratic term, $x^\top \Sigma^{-1} x$, is the same for every class when Σ is shared, so it cancels in every comparison. The word *linear* in LDA describes this induced boundary, not the Gaussian density itself, which is anything but linear.

Proposition 4.4.8 (QDA: class-specific covariance gives quadratic scores). *With separate covariances, up to a term common to all classes,*

$$s_{\hat{\phi},c}(x) = -\frac{1}{2}(x - \hat{\mu}_c)^\top \hat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log |\hat{\Sigma}_c| + \log \hat{\pi}_c.$$

The first term is a squared Mahalanobis distance to the class centre; the determinant term penalises diffuse classes. The class-dependent quadratic terms make the boundaries curved (conic sections in two dimensions). This is Quadratic Discriminant Analysis.

Proof. Same log density as above; now $\log |\hat{\Sigma}_c|$ and the quadratic form both depend on c , and only $-\frac{d}{2} \log(2\pi)$ is common. Equality of two such scores is a quadratic equation in x . $\square$

Reading the formula

LDA versus QDA is a decision about *how many numbers to estimate*. LDA: one shared covariance, $d(d+1)/2$ numbers, plus C means. QDA: one covariance per class, $C \cdot d(d+1)/2$ numbers. In two dimensions LDA can only draw straight lines between classes; QDA can draw ellipses, parabolas and hyperbolas. More flexibility, more data required. The geometry of the boundary is the visible fingerprint of the assumption you made about the class clouds.

Proposition 4.4.9 (Parameter count of a covariance). *A symmetric $d \times d$ matrix has $d(d+1)/2$ distinct entries. For $d = 100$ that is 5050 per class.*

Proof. The diagonal has d entries; the strictly upper triangle has $\binom{d}{2} = d(d-1)/2$, and symmetry fixes the lower triangle. Total $d + d(d-1)/2 = d(d+1)/2$. With $d = 100$: $100 \cdot 101/2 = 5050$. $\square$

If a class has few observations, its covariance estimate may be unstable or singular (not invertible, so the score cannot even be computed). Two responses: regularise the estimate, or impose stronger structure.

Definition 4.4.10 (Covariance shrinkage). $\hat{\Sigma}_\lambda = (1-\lambda)\hat{\Sigma} + \lambda\alpha I$ for $0 \leq \lambda \leq 1$ and a reference variance $\alpha > 0$ (e.g. the average diagonal entry). $\lambda = 0$ keeps the empirical covariance; $\lambda = 1$ removes all correlations and gives every

direction the same variance. Validation selects λ .

Reading the formula

Shrinkage blends the noisy, data-specific matrix with a stable, featureless one. For any $\lambda > 0$ the result is positive definite (a positive-definite matrix plus a positive multiple of the identity), so it can always be inverted. Taking $\lambda \rightarrow 1$ and then dropping the remaining correlations altogether leads to the next section.

Technique: Gaussian classifiers by hand

- Compute class counts, means, and variances (divide by N_c).
- Drop constants common to all classes before comparing: with equal variances compare $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2 / (2\sigma^2)$ only.
- To get a posterior, exponentiate the scores and normalise (softmax of the generative scores, Definition 4.4.6).
- Equal priors and equal variances in one dimension: the boundary is the midpoint of the means. A larger prior moves it away from that class; a larger variance in QDA bends the region.

Classic problems

- *Two classes, equal priors, $\mu_0 = 0$, $\mu_1 = 4$, shared $\sigma^2 = 1$. Boundary?* $x = 2$.
- *Same, but $\pi_0 = 0.9$, $\pi_1 = 0.1$. Solve $-\frac{1}{2}x^2 + \log 0.9 = -\frac{1}{2}(x-4)^2 + \log 0.1$:* $4x - 8 = \log 9 \approx 2.197$, so $x \approx 2.55$.
- *$d = 10$ features, QDA with $C = 3$ classes: how many covariance parameters?* $3 \times 55 = 165$; LDA: 55.
- *Class with $N_c = 5$ examples in $d = 10$ dimensions: what goes wrong?* The 10×10 covariance has rank at most 4: singular, not invertible. Shrink or use Naive Bayes.

Summary: Gaussian class models

- MLE in closed form: class frequencies, class means, class covariances with denominator N_c (Theorem 4.4.2).
- Fitted posterior = softmax of generative scores $\log \hat{\pi}_c + \log p(x | y = c)$.
- Shared covariance $\Rightarrow$ affine scores $\Rightarrow$ hyperplane boundaries (LDA, Theorem 4.4.7); separate covariances $\Rightarrow$ quadratic boundaries (QDA).
- A covariance costs $d(d+1)/2$ parameters; shrink it or simplify the model when data are scarce.

4.5 Naive Bayes

What are we trying to get?

When estimating a full joint distribution of the features is hopeless, we can pretend the features are independent *within each class*. We want to see exactly what this buys (one small model per feature, additive evidence, stability on little data), what it costs (double-counted evidence, overconfident probabilities), and how to fix the two failure modes that appear in practice: zero variances and zero counts.

Definition 4.5.1 (Naive Bayes). Naive Bayes assumes the features are *conditionally independent given the class*:

$$p_{\theta}(x \mid y = c) = \prod_{j=1}^d p_{\theta}(x_j \mid y = c).$$

Taking logarithms, the generative score becomes *additive*:

$$s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \sum_{j=1}^d \log p_{\hat{\theta}}(x_j \mid y = c).$$

Reading the formula

Read the score as a ledger: start with the prior evidence $\log \hat{\pi}_c$, then add one line of evidence per feature, each computed from a *one-dimensional* model of that feature within class c . Flow duration, packet counts and byte statistics may in reality be correlated within an attack class, so the assumption is strong. What it buys is that a difficult d -dimensional density estimation problem becomes d easy one-dimensional ones, each of which can be stable with limited data.

Interpretation & discussion

Two facts make Naive Bayes far more useful than its assumption suggests. First, a *misspecified density can still rank classes correctly*: getting the largest score right is a weaker requirement than getting every density right. Second, the one-dimensional models need not be Gaussian or even of the same family: a positive skewed feature may get a Gamma or log-normal, a count a Poisson, an irregular feature a histogram. The main risk is that *correlated features repeat the same evidence*: two nearly identical features count twice, so the classifier may still classify well while its probabilities become too extreme. Removing redundant features improves calibration.

Gaussian Naive Bayes and variance smoothing

For numerical features, Gaussian Naive Bayes models $x_j \mid (y = c) \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}_{cj}^2)$, with the one-dimensional MLEs of Theorem 4.4.2 applied feature by feature. Two problems arise. If a fitted variance is 0 (all class- c values of feature j coincide), the density is undefined. If it is merely tiny, that one feature's narrow density dominates the whole class score.

Definition 4.5.2 (Variance smoothing). Replace each fitted variance by $\tilde{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ with $\lambda > 0$, where s_j^2 is a reference scale such as the overall training variance of feature j . A small λ prevents numerical instability; if its value materially affects predictions, select it on validation data.

Example 4.5.3 (A tiny variance takes over). Two features, class c : feature 1 has $\hat{\sigma}^2 = 1$, feature 2 has $\hat{\sigma}^2 = 10^{-6}$ (every training example of class c had exactly the same value, say a port number). A test point whose feature 2 differs from that value by 0.01 contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score of class c , dwarfing anything feature 1 can say. With $\lambda s_2^2 = 0.01$ added, the contribution becomes $-\frac{1}{2}(10^{-4})/(0.01) \approx -0.005$: feature 2 still counts, but no longer vetoes.

Variant	Feature model	Typical use
Gaussian	continuous Gaussian variable	measurements, flow statistics
Bernoulli	binary presence/absence	indicators
Multinomial	nonnegative event or token counts	documents, event counts
Hybrid	distribution chosen by feature type	mixed tabular data

A hybrid model multiplies contributions from different feature-specific distributions under one class prior; the prior appears once in the score, never once per feature.

Multinomial Naive Bayes for text

Definition 4.5.4 (Multinomial Naive Bayes). Fix a vocabulary of V words and represent a document by its count vector $x = (x_1, \dots, x_V)$. For class c , let $\theta_{cj} = p_\theta(\text{token} = j \mid y = c)$ with $\sum_j \theta_{cj} = 1$. Ignoring the multinomial coefficient (common to all classes),

$$p_\theta(x \mid y = c) \propto \prod_{j=1}^V \theta_{cj}^{x_j},$$

with the convention $0^0 = 1$. Repeated words contribute repeated evidence; absent words ($x_j = 0$) contribute the factor 1; word order is ignored.

Reading the formula

Imagine class c as a bag of words with proportions θ_c ; a document is produced by drawing its words from the bag. “free free prize” with vocabulary (free, win, prize, meeting, project, report) has $x = (2, 0, 1, 0, 0, 0)$ and likelihood $\propto \theta_{c,\text{free}}^2 \theta_{c,\text{prize}}$. The same word twice is twice the evidence. The multinomial coefficient (how many orderings produce these counts) is the same for every class and cancels in Bayes’ rule.

Example 4.5.5 (Why smoothing is needed). Aggregate training counts, seven tokens per class:

	free	win	prize	meeting	project	report
spam	3	2	2	0	0	0
ham	0	0	0	3	2	2

Without smoothing, $\hat{\theta}_{cj} = n_{cj}/7$. The test message “free meeting” gives

$$p(x \mid \text{spam}) \propto \frac{3}{7} \cdot \frac{0}{7} = 0, \quad p(x \mid \text{ham}) \propto \frac{0}{7} \cdot \frac{3}{7} = 0.$$

One unseen word zeroes an entire product, and here *both* products vanish, so the posterior 0/0 is undefined. A single word never seen in training in a class would otherwise veto that class no matter how strong the other evidence.

Definition 4.5.6 (Additive smoothing). With n_{cj} the training count of token j in class c and $\alpha > 0$,

$$\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell=1}^V n_{c\ell} + \alpha V}.$$

For a categorical feature x_j with K_j levels, with n_{c_jv} the count of value v in class c and $n_{cj} = \sum_v n_{c_jv}$,

$$p_{\hat{\theta}}(x_j = v \mid y = c) = \frac{n_{c_jv} + \alpha}{n_{cj} + \alpha K_j}.$$

Proposition 4.5.7 (Smoothed estimates are proper and interpolate to uniform). For every $\alpha > 0$: (i) $\hat{\theta}_{cj} > 0$ for all j and $\sum_j \hat{\theta}_{cj} = 1$; (ii) as $\alpha \rightarrow 0$, $\hat{\theta}_{cj} \rightarrow n_{cj} / \sum_{\ell} n_{c\ell}$ (the unsmoothed estimate); (iii) as $\alpha \rightarrow \infty$, $\hat{\theta}_{cj} \rightarrow 1/V$ (uniform). Moreover $\hat{\theta}_{cj}$ is the estimate one obtains by adding α imaginary occurrences of every word to the class.

Proof. (i) Numerators are $\geq \alpha > 0$; summing $n_{cj} + \alpha$ over j gives exactly the denominator. (ii) and (iii): divide numerator and denominator by α for (iii), or set $\alpha = 0$ in the limit for (ii). The last claim is the formula itself: counts $n_{cj} + \alpha$ over total $\sum_{\ell} n_{c\ell} + V\alpha$. $\square$

Reading the formula

Pretend you saw every word α extra times in every class. With $\alpha = 1$ (“Laplace smoothing”) and $V = 6$: spam counts become (4, 3, 3, 1, 1, 1) over $7 + 6 = 13$, ham counts (1, 1, 1, 4, 3, 3) over 13:

$$\hat{\theta}_{\text{spam}} = \frac{1}{13}(4, 3, 3, 1, 1, 1), \quad \hat{\theta}_{\text{ham}} = \frac{1}{13}(1, 1, 1, 4, 3, 3).$$

Larger α pulls all estimates toward $1/6$; smaller α trusts the counts more.

Example 4.5.8 (The smoothed verdict on “free meeting”). With equal priors $\frac{1}{2}$, the unnormalised scores are

$$\exp[s_{\text{spam}}] = \frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13} = \frac{2}{169}, \quad \exp[s_{\text{ham}}] = \frac{1}{2} \cdot \frac{1}{13} \cdot \frac{4}{13} = \frac{2}{169},$$

so $p(\text{spam} \mid x) = \frac{1}{2}$. Both words are now possible under both classes, and in

this symmetric message their evidence balances exactly.

Example 4.5.9 (A less symmetric message: “free prize”). $\exp[s_{\text{spam}}] = \frac{1}{2} \cdot \frac{4}{13} \cdot \frac{3}{13} = \frac{12}{338}$ and $\exp[s_{\text{ham}}] = \frac{1}{2} \cdot \frac{1}{13} \cdot \frac{1}{13} = \frac{1}{338}$, so $p(\text{spam} | x) = 12/13 \approx 0.92$. And “free free prize” (the word *free* twice): $\frac{1}{2}(\frac{4}{13})^2 \frac{3}{13} = \frac{48}{4394}$ versus $\frac{1}{2}(\frac{1}{13})^2 \frac{1}{13} = \frac{1}{4394}$, so $p(\text{spam} | x) = 48/49 \approx 0.98$: the repeated word counts twice, as the multinomial model says it should.

Remark 4.5.10 (Unseen categories). A categorical value that appears at validation or test time but never in training (a new port number, say) must have a defined representation: create an *other* level during preprocessing, include it among the K_j levels, and map every unseen value to it. Smoothing then gives it a small positive probability instead of a crash.

Technique: Naive Bayes by hand

- Work in logs when there are more than three or four features; products of small numbers underflow and are error-prone.
- Apply smoothing *before* normalising: add α to every count, add αV (or αK_j) to the total.
- Compare unnormalised scores first; normalise only if a posterior is asked for.
- Count the prior exactly once, not once per feature.
- Remove near-duplicate features (e.g. bytes and packets when they are proportional) to avoid double counting.

Classic problems

- $V = 4$, class counts $(5, 0, 3, 2)$, $\alpha = 1$. Smoothed probabilities? $(6, 1, 4, 3)/14$.
- Same counts, $\alpha = 100$: roughly? Close to $(1, 1, 1, 1)/4$: $(105, 100, 103, 102)/410 \approx (0.256, 0.244, 0.251, 0.249)$.
- Bernoulli NB with features (has link, from known sender): spam rates $(0.9, 0.1)$, ham rates $(0.2, 0.8)$, priors 0.5. An email with a link from a known sender? Spam: $0.5 \cdot 0.9 \cdot 0.1 = 0.045$; ham: $0.5 \cdot 0.2 \cdot 0.8 = 0.08$. Ham, with posterior $0.08/0.125 = 0.64$.
- Why can Gaussian NB have a boundary that is not linear even though each feature is Gaussian? Class-specific variances per feature make the scores quadratic in x (a diagonal QDA).

Summary: Naive Bayes

- Assume conditional independence of features given the class; the score becomes prior evidence plus one additive term per feature.
- One-dimensional models per feature: Gaussian, Bernoulli, multinomial, or a hybrid; choose by feature type and validate.
- Fix zero variances by adding λs_j^2 ; fix zero counts by additive smoothing

$(n + \alpha)/(\text{total} + \alpha V)$ (Proposition 4.5.7).

- Correlated features double-count evidence: ranking may survive, calibration suffers.

4.6 Comparing the Two Routes

What are we trying to get?

We want a fair statement of when each route is preferable, phrased in terms of assumptions and data, not slogans.

Question	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y \mid x)$ or a boundary	prior π_c and class model $p_\theta(x \mid y = c)$
Input assumptions	usually fewer	usually stronger
Primary objective	predict the class	describe class-specific inputs
Missing features	need separate treatment	can sometimes be marginalised
Sampling inputs	usually unavailable	possible in principle
Raw high-dimensional input	often practical	full density estimation is hard

Interpretation & discussion

Generative structure can *reduce estimation error*: a restricted description of each class has few parameters, so with little labelled data it can be fitted reliably, provided the assumptions are approximately right. A wrong class-conditional family creates *approximation bias* that does not vanish with more data. A sufficiently flexible discriminative model may eventually reach lower classification error because it spends no capacity modelling aspects of X that are irrelevant to Y . Schematically, error versus labelled data often shows the generative curve lower at small N and the discriminative curve lower at large N , but this is not a universal law: the crossing point, and whether it exists, depends on the data distribution, the two families, regularisation and optimisation. The right question is not “which route is better” but “which assumptions are credible, and which deployment objective matters.”

Example 4.6.1 (Same data, two verdicts). Spam data with 200 labelled emails and a 10,000-word vocabulary. Multinomial Naive Bayes needs to estimate $2 \times 10,000$ smoothed word probabilities, each from simple counts: it works reasonably from day one. Logistic regression on the same counts must locate a 10,000-dimensional hyperplane from 200 points: badly underdetermined, prone to overfitting. With 2 million labelled emails, the picture flips: logistic regression (or a network) learns interactions such as “*free* is spammy unless *invoice* is nearby,” which the independence assumption forbids Naive Bayes from ever representing.

Summary: Comparing the routes

- Generative: more assumptions, less data needed, persistent bias if wrong, extra abilities (missing features, sampling, plausibility).
- Discriminative: fewer assumptions, better with much data, nothing about the input distribution.
- Decide by credibility of assumptions and by the deployment objective, then validate.

4.7 Class Imbalance and Prior Shift

What are we trying to get?

Two situations are constantly confused: a class that is rare, and a class whose frequency *changes* between training and deployment. We want to separate them, and for the second, derive the exact correction for both routes.

Definition 4.7.1 (Calibration). A probabilistic classifier is *calibrated* if, among cases assigned probability u , approximately a fraction u are positive. Calibration concerns the interpretation of probabilities; the choice of a decision threshold from costs concerns the action taken from them. These are separate questions.

If training and deployment share the same joint distribution, an imbalanced training prior is not an error: the fitted posterior should reflect that prevalence. The difficulty is finite minority data: with few positives, the minority distribution or the boundary is poorly estimated, and weighting or resampling may help estimation. Otherwise their benefit is limited and the resulting probabilities may need recalibration.

Definition 4.7.2 (Class-prior shift). Between training (t) and deployment (d), the class-conditional distributions are unchanged but the priors differ:

$$p_d(x | y = c) = p_t(x | y = c), \quad \pi_{d,c} \neq \pi_{t,c}.$$

Example: attacks form 20% of the training data but 5% of deployment traffic.

Theorem 4.7.3 (Generative correction). *Under prior shift, the deployment posterior is obtained by keeping the fitted class-conditional models and replacing the training priors by the deployment priors:*

$$p_d(y = c | x) = \frac{\pi_{d,c} p_{\hat{\theta}}(x | y = c)}{\sum_{k=1}^C \pi_{d,k} p_{\hat{\theta}}(x | y = k)}.$$

No class-conditional model needs to be refitted.

Proof. Bayes' rule (Theorem 4.3.2) at deployment uses π_d and $p_d(x | y)$; by Definition 4.7.2 the latter equals $p_t(x | y)$, which is what the fitted $p_{\hat{\theta}}(x | y)$ estimates. Substituting gives the formula. $\square$

Theorem 4.7.4 (Discriminative correction, binary case). Let $r_t(x) = p_t(y = 1 \mid x)$ and $r_d(x) = p_d(y = 1 \mid x)$, and write $\pi_t = p_t(y = 1)$, $\pi_d = p_d(y = 1)$. Under prior shift,

$$r_d(x) = \frac{(\pi_d/\pi_t) r_t(x)}{(\pi_d/\pi_t) r_t(x) + \frac{1 - \pi_d}{1 - \pi_t} [1 - r_t(x)]}.$$

Proof. Step 1 (odds form of Bayes' rule). At training time, $\frac{r_t(x)}{1 - r_t(x)} = \frac{\pi_t p(x \mid y = 1)}{(1 - \pi_t) p(x \mid y = 0)}$: divide the two numerators of Theorem 4.3.2; the common denominator cancels. Likewise at deployment with π_d .

Step 2 (the likelihood ratio is shared). Under prior shift, $p(x \mid y = 1)/p(x \mid y = 0)$ is the same at both times. Solving the training odds for it and substituting into the deployment odds,

$$\frac{r_d(x)}{1 - r_d(x)} = \frac{\pi_d/(1 - \pi_d)}{\pi_t/(1 - \pi_t)} \cdot \frac{r_t(x)}{1 - r_t(x)}.$$

Step 3 (solve for r_d). If odds = A/B then probability = $A/(A + B)$. With $A = \frac{\pi_d}{\pi_t} r_t(x)$ and $B = \frac{1 - \pi_d}{1 - \pi_t} (1 - r_t(x))$, the odds above equal A/B , hence $r_d = A/(A + B)$, which is the displayed formula. $\square$

Reading the formula

The correction *reweights the two hypotheses*: multiply the training probability of class 1 by “how much rarer class 1 became” (π_d/π_t), multiply the probability of class 0 by “how much commoner class 0 became” ($(1 - \pi_d)/(1 - \pi_t)$), then renormalise. Attacks 20% $\rightarrow$ 5%: $\pi_d/\pi_t = 0.25$ and $(1 - \pi_d)/(1 - \pi_t) = 0.95/0.80 = 1.1875$. A flow with training posterior $r_t = 0.5$ gets $A = 0.125$, $B = 0.59375$, so $r_d = 0.125/0.71875 \approx 0.17$. What looked like a coin flip in training is a 17% attack probability in deployment, because attacks are four times rarer there. The correction assumes r_t is a *calibrated* training posterior; the generative route makes the prior an explicit component, the discriminative route can only correct after the fact and depends on the quality of its probabilities.

Gotcha

- Imbalance $\neq$ shift. Imbalance with enough minority data needs no correction at all; shift needs one even with abundant data.
- Reweighting or resampling the training set *changes the effective prior*: the fitted posterior then corresponds to the resampled prior and must be corrected back with Theorem 4.7.4 if calibrated probabilities are wanted.
- Cost-sensitive thresholds (Theorem 2.2.9) and prior correction are different operations; apply the correction first, then the threshold.

Summary: Imbalance and prior shift

- Rare class with enough data: the posterior should reflect the prevalence; with too little data, weighting or resampling helps estimation but may hurt calibration.

- Prior shift: same class-conditionals, new priors. Generative: swap the priors (Theorem 4.7.3). Discriminative: reweight the odds by π_d/π_t and $(1 - \pi_d)/(1 - \pi_t)$ (Theorem 4.7.4), which requires calibrated training probabilities.

4.8 Unfamiliar Inputs and Combined Systems

What are we trying to get?

Security systems meet inputs that belong to no known class. We want a mechanism that can say “I do not recognise this,” and to see how modern systems combine the two routes, including generative language models used for classification.

Posterior confidence is not familiarity

A high value of $\max_c p_{\hat{\theta}}(y = c | x)$ only means that one modelled class outranks the others. It does not establish that x resembles the training data. A new attack mechanism may lie far from every known class while still receiving a confident known-class prediction.

Definition 4.8.1 (Reject option via a compatibility check). Choose the most probable known class $c^* = \arg \max_c p_{\hat{\theta}}(y = c | x)$, then compare the fitted class likelihood with a class-specific threshold:

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\hat{\theta}}(x | y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise.} \end{cases}$$

Reading the formula

Two questions instead of one. *Which known class fits best?* (the posterior). *Does even the best class fit well enough?* (the likelihood against a threshold). Only a generative model can answer the second, because only it has a notion of what inputs of class c look like. The threshold τ_c should be validated on data that reproduce the intended novelty, e.g. attack mechanisms excluded from training. A low likelihood is evidence of incompatibility, not a guarantee of detection: high-dimensional likelihoods behave counterintuitively, and a misspecified class model may give low likelihood to perfectly valid cases.

Example 4.8.2 (Combining both routes). A discriminative model ranks the known classes for an observed flow, producing $p_{\hat{\theta}_d}(y = c | x)$. A generative or anomaly model computes $\log p_{\hat{\theta}_g}(x | y = c)$ for the predicted class. The final policy accepts the known class, requests human review, or rejects the input as unfamiliar. The discriminative model is good at the ranking; the generative model is the plausibility check it lacks.

Generative language models and classification

An autoregressive language model is trained *generatively* by factorising a token sequence,

$$p_{\theta}(w_{1:T}) = \prod_{t=1}^T p_{\theta}(w_t \mid w_{<t}),$$

its pretraining objective being next-token prediction (Chapter 1, Example 1.5.4). A *sequence-classification head* instead models $p_{\psi}(y \mid w_{1:T})$ and minimises the discriminative objective $L_{\text{disc}}(\psi) = -\sum_i \log p_{\psi}(y^{(i)} \mid w_{1:T_i}^{(i)})$: one label for the entire sequence, not one per token. Alternatively, a prompt can ask the model to generate or score label tokens directly. A single system thus combines generative pretraining with a discriminative downstream objective.

Summary: Unfamiliar inputs and combined systems

- Softmax confidence compares known classes only; a reject option needs a likelihood check (Definition 4.8.1) with validated thresholds.
- Modern systems combine a discriminative ranker with a generative compatibility model.
- Language models: generative pretraining (next token) plus a discriminative classification head or prompted label scoring.

4.9 Model Choice and Lab Preparation

Three questions organise model choice.

1. *Can a credible class-conditional model $p_{\theta}(x \mid y)$ be stated?* If so, its structure may reduce the amount of labelled data required.
2. *Is the main objective accurate prediction with many labelled examples?* A flexible discriminative model avoids unnecessary assumptions about the input distribution.
3. *Is the raw input complex and high-dimensional?* Learn a representation first (Chapter 1), then compare simple classification heads; that is more credible than modelling the raw density.

Deployment-like validation decides between credible alternatives. The labels “generative” and “discriminative” do not replace empirical comparison under the intended use conditions.

Example 4.9.1 (Lab 2 at a glance). The lab compares the routes on cybersecurity flow data: Gaussian, multinomial and hybrid Naive Bayes for numerical, service and mixed feature sets, against logistic regression and a decision tree; then limited training data (20% of each class), deployment prior shift ($\pi_d = 0.05$, corrected by swapping priors for the generative models and by Theorem 4.7.4 for the discriminative ones), and multiclass classification with and without destination-port information. Every model must use the same fixed stratified split and the same rows, so that differences reflect the models rather than the data.

Notes on the original slides and notes

All formulas and numbers were rederived: the rare-attack posterior 0.67, the one-dimensional Gaussian example ($\hat{\sigma}^2 = 2/3$ for both classes, posterior 0.950), the covariance count 5050, the smoothed word probabilities and the $\frac{1}{2}$ posterior for “free meeting,” and the prior-shift formula. All are correct. Points to note:

- **The slide deck is duplicated.** The Lecture 4 PDF contains its first 34 slides twice: pages 35–68 repeat pages 1–34 verbatim (title, objectives, two routes, discriminative route, Gaussian models, variance smoothing). The genuinely new material resumes at page 69 (“Beyond Gaussian Naive Bayes”). Nothing is lost, but do not look for new content in the repeated block.
- **Cross-entropy table (notes Section 2.4, slide 14).** “0.9, 0.5, 0.1 $\rightarrow$ 0.11, 0.69, 2.30” uses natural logarithms: $-\ln 0.9 = 0.105$, $-\ln 0.5 = 0.693$, $-\ln 0.1 = 2.303$.
- **Covariance MLE denominator (notes Section 4.1).** The notes correctly stress N_c rather than $N_c - 1$. The proof of the one-dimensional case is Theorem 4.4.2; the multivariate case is stated without the matrix calculus.
- **LDA score (notes eq. 21, slide 28).** The score is affine in x only after dropping $x^\top \Sigma^{-1} x$ and $\log |\Sigma|$, both common to all classes *because* the covariance is shared; Theorem 4.4.7 shows the cancellation explicitly.
- **Shrinkage (notes eq. 24).** The reference variance α is unspecified in the notes; any positive scale works, and the average diagonal entry of $\hat{\Sigma}$ is a common choice. The point is positive-definiteness for any $\lambda > 0$.
- **Multinomial likelihood (notes eq. 29).** The proportionality hides the multinomial coefficient, which cancels in Bayes’ rule because it depends only on x , not on the class. The convention $0^0 = 1$ is needed for absent words.
- **Prior-shift correction (notes eq. 34).** The formula is right; the derivation through odds is Theorem 4.7.4. It silently requires r_t to be calibrated, as the notes remark afterwards.

Chapter takeaways

Summary: Discriminative and generative classification

- **Decisions** depend on the posterior and the loss; the posterior must be *learned*.
- **Discriminative** models estimate $p(y \mid x)$: scores, sigmoid/softmax, cross-entropy as conditional maximum likelihood; the score family fixes the boundary geometry.
- **Generative** classifiers model $p(x \mid y)$ and $p(y)$, then apply Bayes’ rule; the likelihood splits by class and the MLE prior is the class frequency.
- **Gaussian** class models: closed-form MLE; shared covariance gives linear boundaries (LDA), separate covariances quadratic ones (QDA); covariance costs $d(d+1)/2$ parameters.
- **Naive Bayes** trades a strong independence assumption for stability; smooth variances and counts.

- **Trade-off:** generative structure helps with little data if credible; misspecification is persistent; flexible discriminative models win with much data.
- **Imbalance** is an estimation issue; **prior shift** needs a posterior correction (swap priors, or reweight odds).
- **Confidence is not familiarity:** add a likelihood-based reject option; combine routes; language models pair generative pretraining with discriminative heads.